

the current class since you know that the method can't be used  
.anywhere else

By making methods private, you underscore the importance of the public  
.interface of the class and of the methods that remain public

How to Refactor  
Regularly try to find methods that can be made private. Static code  
.analysis and good unit test coverage can offer a big leg up  
  
.Make each method as private as possible

## Hide Method 🐒

### مشکل (Problem)

یه متد توسط کلاس‌های دیگه استفاده نمیشه یا فقط توی سلسله مراتب خودش استفاده میشه.

### راه‌حل (Solution)

متد رو `private` یا `protected` کن.

## 🎯 چرا ری‌فکتور کنیم؟ (Why Refactor)

| توضیح                                                                                   | دلیل              |
|-----------------------------------------------------------------------------------------|-------------------|
| با اضافه شدن رفتار جدید، getter/setter های قدیمی دیگه لازم نیستن                        | توسعه رابط غنی‌تر |
| اگه getter/setter رو <code>private</code> کنی و دسترسی مستقیم بذاری، می‌تونی حذفشون کنی | حذف متدهای اضافی  |

مزایا (Benefits)

| مزیت                     | توضیح                                                 |
|--------------------------|-------------------------------------------------------|
| تکامل آسون تر کد         | با تغییر متد private فقط نگران یه کلاس هستی           |
| تأکید بر Interface عمومی | متدهای private اهمیت Interface عمومی رو برجسته می کنن |

How to Refactor (چطور انجام بدیم)

| قدم | توضیح                                            |
|-----|--------------------------------------------------|
| ۱   | مرتباً دنبال متدهایی بگرد که می تونن private بشن |
| ۲   | هر متد رو تا حد امکان private کن                 |

مثال ساده

قبل:

csharp

```
public class Customer
{
    public string Name { get; set; }
    public string Email { get; set; }

    public bool IsValidEmail() // فقط داخل خود Customer استفاده میشه
    {
        return Email.Contains("@");
    }
}
```

بعد:

csharp

```

        public class Customer
        {
            public string Name { get; set; }
            public string Email { get; set; }

            // مخفی شد
            private bool IsValidEmail()
            {
                return Email.Contains("@");
            }

            public void SendNewsletter()
            {
                if (IsValidEmail())
                {
                    Console.WriteLine($"Sending to {Email}")
                }
            }
        }
    }

```

## تمرین

این کد رو با **Hide Method** اصلاح کن:

csharp

```

        public class BankAccount
        {
            public decimal Balance { get; private set; }

            public void Withdraw(decimal amount)
            {
                if (HasSufficientBalance(amount))
                {
                    Balance -= amount;
                }
            }

            // فقط توی Withdraw استفاده میشه!
            public bool HasSufficientBalance(decimal amount)
            {
                return Balance >= amount;
            }

            // فقط توی همین کلاس استفاده میشه
            public void LogWithdrawal(decimal amount)
            {
            }
        }
    }

```

```

    }
    ;Console.WriteLine($"Withdrawal: {amount}")
    {
        {

```

راهنمایی: HasSufficientBalance و LogWithdrawal فقط داخلی استفاده میشن → private

بفرست تا بررسی کنیم! 🙌

خودت حلش کن

## کد اصلاح شده با Hide Method

csharp

```

public class BankAccount
{
    public decimal Balance { get; private set; }

    public void Withdraw(decimal amount)
    {
        if (HasSufficientBalance(amount))
        {
            ;Balance -= amount
            ;LogWithdrawal(amount)
        }
    }

    private bool HasSufficientBalance(decimal amount)
    {
        ;return Balance >= amount
    }

    private void LogWithdrawal(decimal amount)
    {
        ;Console.WriteLine($"Withdrawal: {amount}")
    }
}

```

| نسخه اصلاح                                        | کد اصلی               |                      |
|---------------------------------------------------|-----------------------|----------------------|
| <input checked="" type="checkbox"/> private       | public                | HasSufficientBalance |
| <input checked="" type="checkbox"/> private       | public                | LogWithdrawal        |
| <input checked="" type="checkbox"/> Withdraw فقط  | شلوغ                  | Interface عمومی      |
| <input checked="" type="checkbox"/> فقط همین کلاس | باید همه جا رو چک کنی | تغییر منطق داخلی     |

## Hide Method تموم شد! 🎉

و با این، Data Class کامل بسته شد:

| وضعیت                                       | درمان                  |
|---------------------------------------------|------------------------|
| <input checked="" type="checkbox"/>         | Encapsulate Field      |
| <input checked="" type="checkbox"/>         | Encapsulate Collection |
| <input checked="" type="checkbox"/> (قبلاً) | Move Method            |
| <input checked="" type="checkbox"/>         | Remove Setting Method  |
| <input checked="" type="checkbox"/>         | Hide Method            |

## Dispensables

| وضعیت                               | بو             |
|-------------------------------------|----------------|
| <input checked="" type="checkbox"/> | Comments       |
| <input checked="" type="checkbox"/> | Duplicate Code |

| وضعیت                                                                             | بو                     |
|-----------------------------------------------------------------------------------|------------------------|
|  | Lazy Class             |
|  | Data Class             |
|  | Dead Code              |
|  | Speculative Generality |

 **Dead Code**؟ متنش رو بفرست!